**Experiment No : 03**

**Aim :**

Learn basics of maplotlib library and its use as visualisation tool in data science pipeline and learn about line plots and scatter plots.

**Theory :**

Matplotlib is a multi-platform data visualization library built on NumPy arrays, and designed to work with the broader SciPy stack.

It was conceived by John Hunter in 2002, originally as a patch to IPython for enabling interactive MATLAB-style plotting via gnuplot from the IPython command line.

One of Matplotlib's most important features is its ability to play well with many operating systems and graphics backends. Matplotlib supports dozens of backends and output types, which means you can count on it to work regardless of which operating system you are using or which output format you wish.

This cross-platform, everything-to-everyone approach has been one of the great strengths of Matplotlib. It has led to a large user base, which in turn has led to an active developer base and Matplotlib's powerful tools and ubiquity within the scientific Python world.

In recent years, however, the interface and style of Matplotlib have begun to show their age. Newer tools like ggplot and ggvis in the R language, along with web visualization toolkits based on D3js and HTML5 canvas, often make Matplotlib feel clunky and old-fashioned.

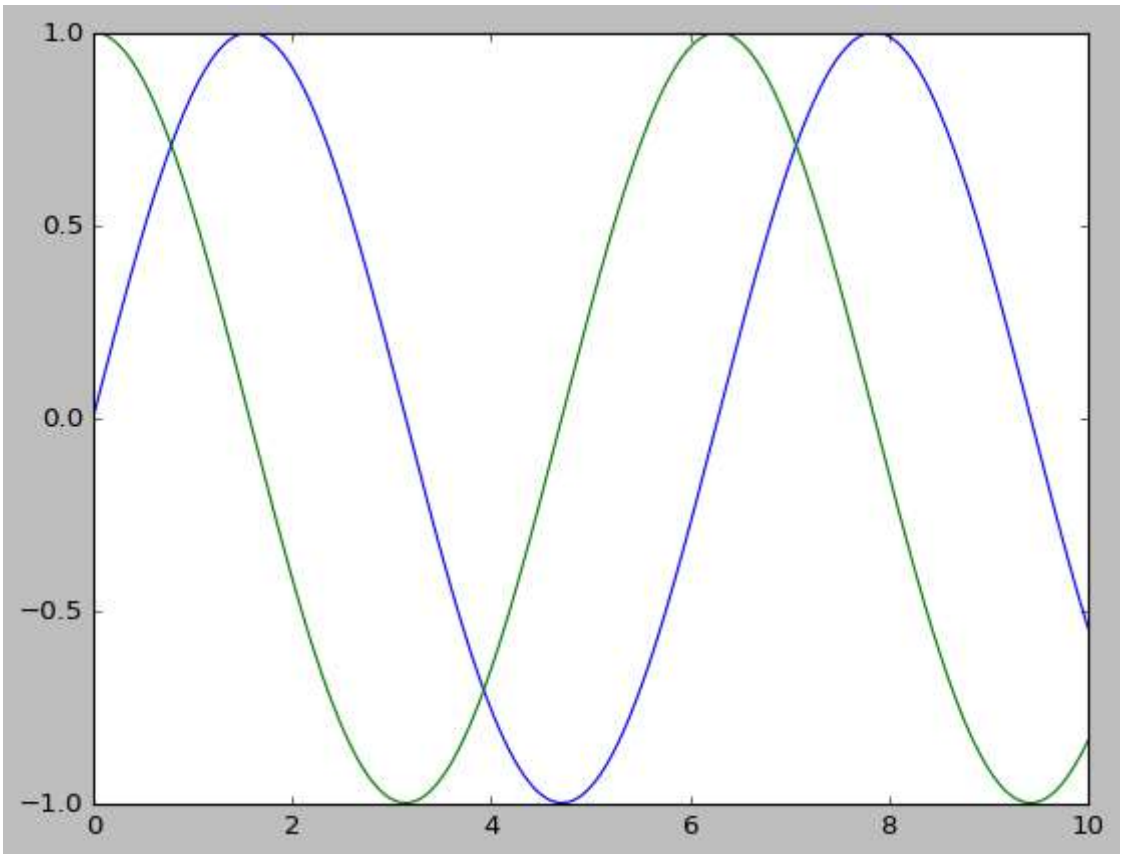
**Performance :**

```
import matplotlib as mpl
import matplotlib.pyplot as plt
mpl.__version__
```

```
'3.10.0'
```

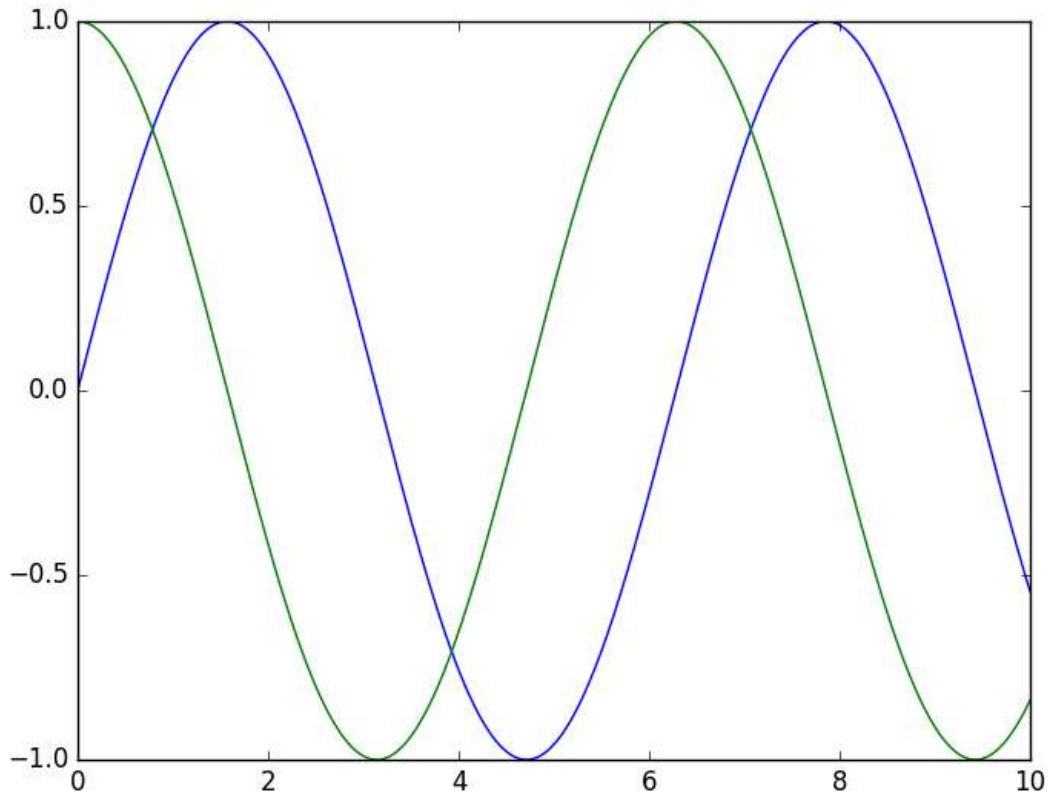
```
#setting ploting style
plt.style.use('classic')
```

```
%matplotlib inline
import numpy as np
x = np.linspace(0, 10, 1000)

fig = plt.figure()
plt.plot(x, np.sin(x))
plt.plot(x, np.cos(x));
```




```
#Save above plot with file named my_figure.png in Current Working Directory
fig.savefig('my_figure.png')
```

```
# use Image package in python to check contents of png file in cwd
from IPython.display import Image
Image('my_figure.png')
```




```
#checking supported file types by matplotlib
fig.canvas.get_supported_filetypes()
```

```
{'eps': 'Encapsulated Postscript',
 'jpg': 'Joint Photographic Experts Group',
 'jpeg': 'Joint Photographic Experts Group',
 'pdf': 'Portable Document Format',
 'pgf': 'PGF code for LaTeX',
 'png': 'Portable Network Graphics',
 'ps': 'Postscript',
 'raw': 'Raw RGBA bitmap',
 'rgba': 'Raw RGBA bitmap',
 'svg': 'Scalable Vector Graphics',
 'svgz': 'Scalable Vector Graphics',
 'tif': 'Tagged Image File Format',
 'tiff': 'Tagged Image File Format',
 'webp': 'WebP Image Format'}
```

**Matplotlib API Interfaces**

A potentially confusing feature of Matplotlib is its dual interfaces: a convenient MATLAB-style state-based interface, and a more powerful object-oriented interface. This section we will review them breifly.

MATLAB-style Interface Matplotlib was originally written as a Python alternative for MATLAB users, and much of its syntax reflects that fact. The MATLAB-style tools are contained in the pyplot (plt) interface. See the example inthe code cell below.

**3:- Create a collection of 100 random normal numbers with parameter mean 30, sd 4 store it in object summer_temp**

ans :- You can create 100 random normal numbers using NumPy like this: import numpy as np

# Create 100 random normal numbers

summer_temp = np.random.normal(loc=30, scale=4, size=100)

print(summer_temp) Explanation:

loc=30 → mean = 30

scale=4 → standard deviation = 4

size=100 → total 100 random values

Stored in object summer_temp

**4:- Sort above set and store in objrct sorted_summer_temp**

import numpy as np

# Create 100 random normal numbers

summer_temp = np.random.normal(loc=30, scale=4, size=100)

# Sort the array

sorted_summer_temp = np.sort(summer_temp)

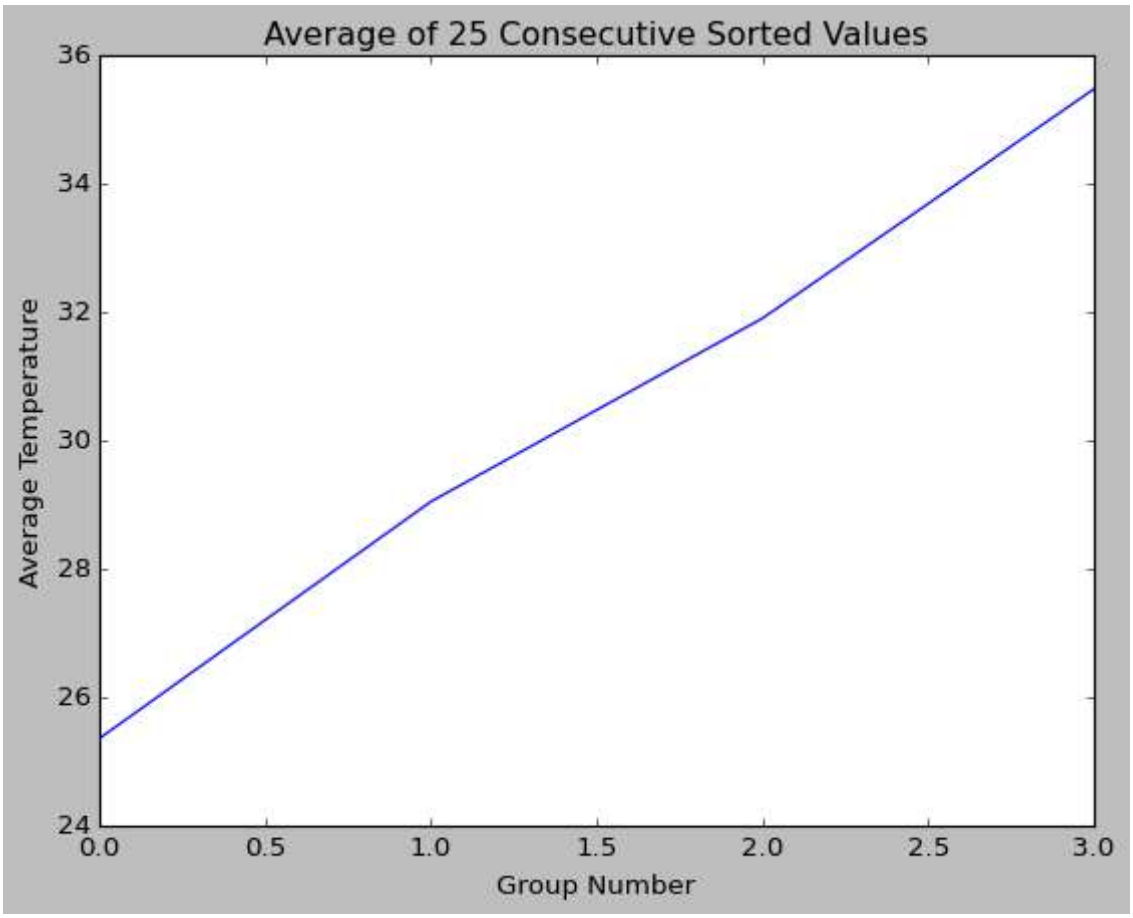
print(sorted_summer_temp) Explanation:

np.sort() → sorts the array in ascending order

Sorted values are stored in sorted_summer_temp

**5 :- Store average of 25 consecutive elements from sorted_summer_temp into object x and plot it as a line plot. What kind of information do u get about the data?**

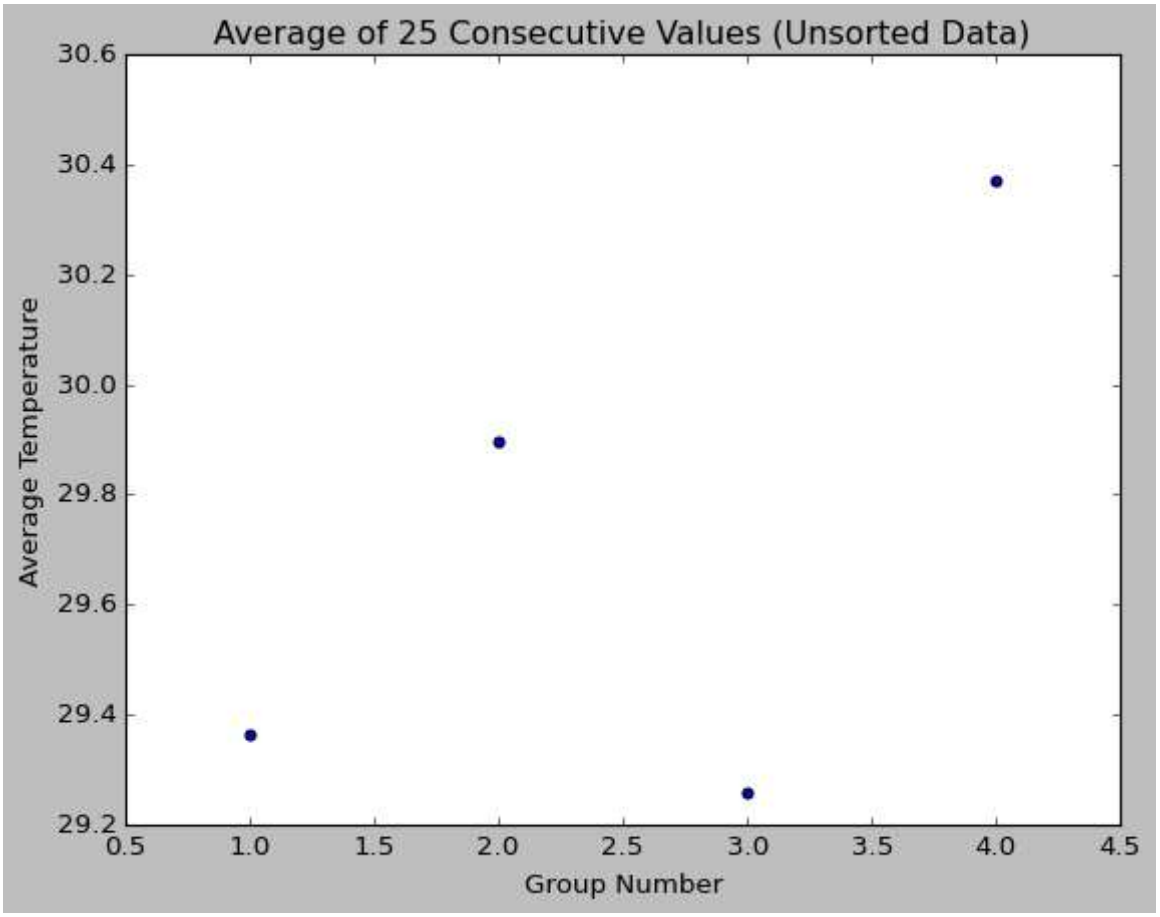
```
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Generate data
summer_temp = np.random.normal(loc=30, scale=4, size=100)

# Step 2: Sort data
sorted_summer_temp = np.sort(summer_temp)

# Step 3: Compute average of every 25 consecutive elements
x = []

for i in range(0, 100, 25):
    avg = np.mean(sorted_summer_temp[i:i+25])
    x.append(avg)

x = np.array(x)

# Step 4: Plot line graph
plt.plot(x)
plt.title("Average of 25 Consecutive Sorted Values")
plt.xlabel("Group Number")
plt.ylabel("Average Temperature")
plt.show()
```



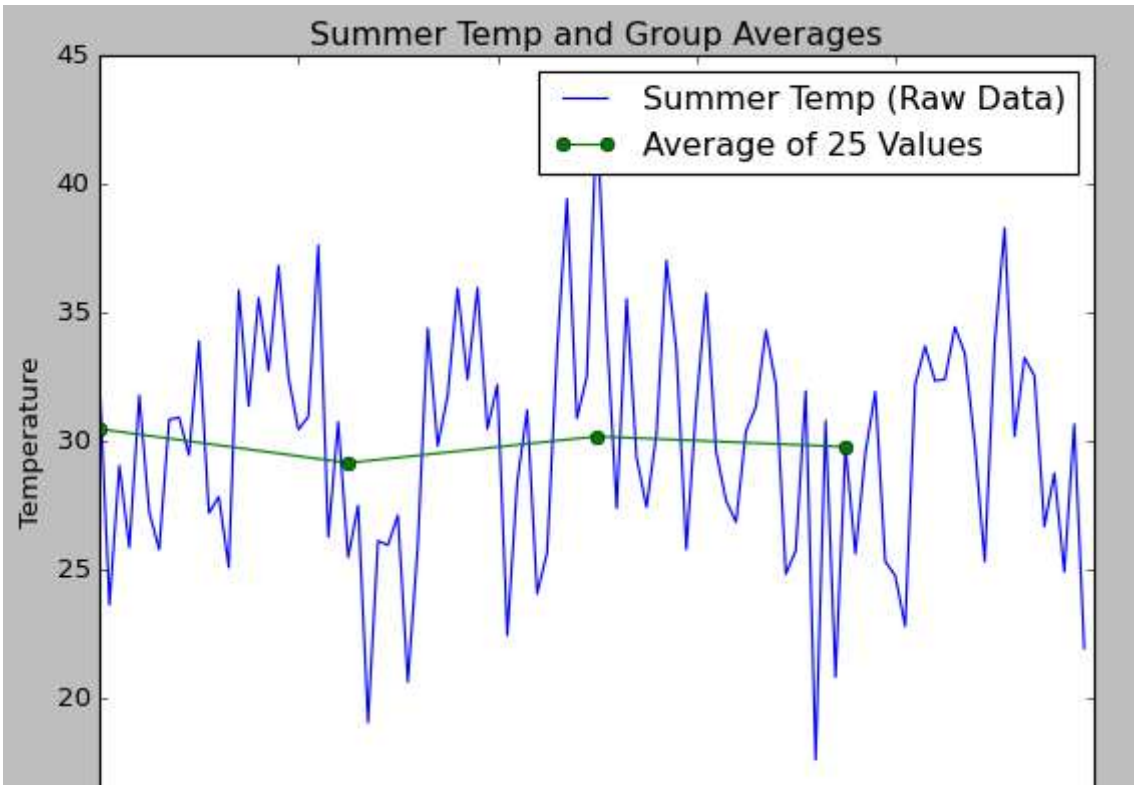

**6:- Repeat above step for summer_temp, store summary in object y and this time use scatter plot to plot values. What kind of information do u get about the data?**

```
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Generate data
summer_temp = np.random.normal(loc=30, scale=4, size=100)

# Step 2: Compute average of every 25 consecutive elements (without sorting)
y = []

for i in range(0, 100, 25):
    avg = np.mean(summer_temp[i:i+25])
    y.append(avg)

y = np.array(y)

# Step 3: Scatter Plot
plt.scatter(range(1, 5), y)
plt.title("Average of 25 Consecutive Values (Unsorted Data)")
plt.xlabel("Group Number")
plt.ylabel("Average Temperature")
```

```
plt.show()
```




**7 :- plot summer_temp and y together in same line plot.**

```
import numpy as np
import matplotlib.pyplot as plt

# Generate data
summer_temp = np.random.normal(loc=30, scale=4, size=100)

# Compute average of every 25 consecutive elements
y = []
for i in range(0, 100, 25):
    avg = np.mean(summer_temp[i:i+25])
    y.append(avg)

y = np.array(y)

# Plot both in same graph
plt.plot(summer_temp, label="Summer Temp (Raw Data)")
plt.plot(range(0, 100, 25), y, marker='o', label="Average of 25 Values")

plt.title("Summer Temp and Group Averages")
plt.xlabel("Index")
```

```
plt.ylabel("Temperature")
plt.legend()
plt.show()
```